



Instituto Tecnológico de Puerto Rico

Recinto de San Juan

LABORATORIO 6 - SEMANA 7

Estructuras y uniones | Del programa en consola a una interfaz visual

Curso	TESI 2105 - Programación Orientada a Objetos
Semana	7
Actividad	Laboratorio 6
Tema	Estructuras y uniones
Valor	50 puntos
Evidencia	Programa de consola + interfaz PyUIBuilder/Tkinter + explicación de conceptos
Propósito: aplicar declaración/agrupación de datos, acceso a miembros y el concepto de unión en un ejemplo sencillo; después visualizar la misma lógica mediante una GUI básica sin adelantar clases y objetos.	
Nombre del estudiante	Juan Antonio Reyes Fernandez
Repositorio / carpeta	https://github.com/JuanReyes1220/TESI2105_Programa_Evolutivo_ReyesJuan.git
Fecha	6 de octubre de 2026
Sección	
1. PARTE A - PROGRAMA INICIAL EN PYTHON	

Crea un archivo llamado semana7_estructuras_union.py. Escribe y ejecuta el siguiente programa. No pases a PyUIBuilder hasta que el programa funcione en la terminal.

```
equipo = {
    "nombre": input("Nombre del equipo: "),
    "ram": int(input("RAM en GB: ")),
    "estado": input("Estado: "),
}

dato_alternativo = {
    "tipo": input("Tipo de identificador (serie/activo): "),
    "valor": input("Valor del identificador: "),
}

print("\nREGISTRO DEL EQUIPO")
print("Nombre:", equipo["nombre"])
print("RAM:", equipo["ram"], "GB")
print("Estado:", equipo["estado"])
print("Identificador:", dato_alternativo["tipo"], "=", dato_alternativo["valor"])
```

Prueba requerida: ejecuta el programa con datos distintos al ejemplo del profesor. Conserva una captura de la ejecución.

```

semana7_estructuras_uniones.py
C: > Users > reyes > Desktop > Bori Style Trabajo Final > semana7_estructuras_uniones.py
1  equipo = {
2      "nombre": input("Nombre del equipo: "),
3      "ram": int(input("RAM en GB: ")),
4      "estado": input("Estado: "),
5  }
6
7  dato_alternativo = {
8      "tipo": input("Tipo de identificador (serie/activo): "),
9      "valor": input("Valor del identificador: "),
10 }
11
12 print("\nREGISTRO DEL EQUIPO")
13 print("Nombre:", equipo["nombre"])
14 print("RAM:", equipo["ram"], "GB")
15 print("Estado:", equipo["estado"])
16 print("Identificador:", dato_alternativo["tipo"], "=", dato_alternativo["valor"])

```

Problems Output Debug Console Terminal Ports

PS C:\Users\reyes\AppData\Local\Programs\Microsoft VS Code> & C:\Users\reyes\eyes\Desktop\Bori Style Trabajo Final\semana7_estructuras_uniones.py

Nombre del equipo: Dell
RAM en GB: 16
Estado: Excelente
Tipo de identificador (serie/activo): 45JH387739
Valor del identificador: 10090

REGISTRO DEL EQUIPO
Nombre: Dell
RAM: 16 GB
Estado: Excelente
Identificador: 45JH387739 = 10090

PS C:\Users\reyes\AppData\Local\Programs\Microsoft VS Code>

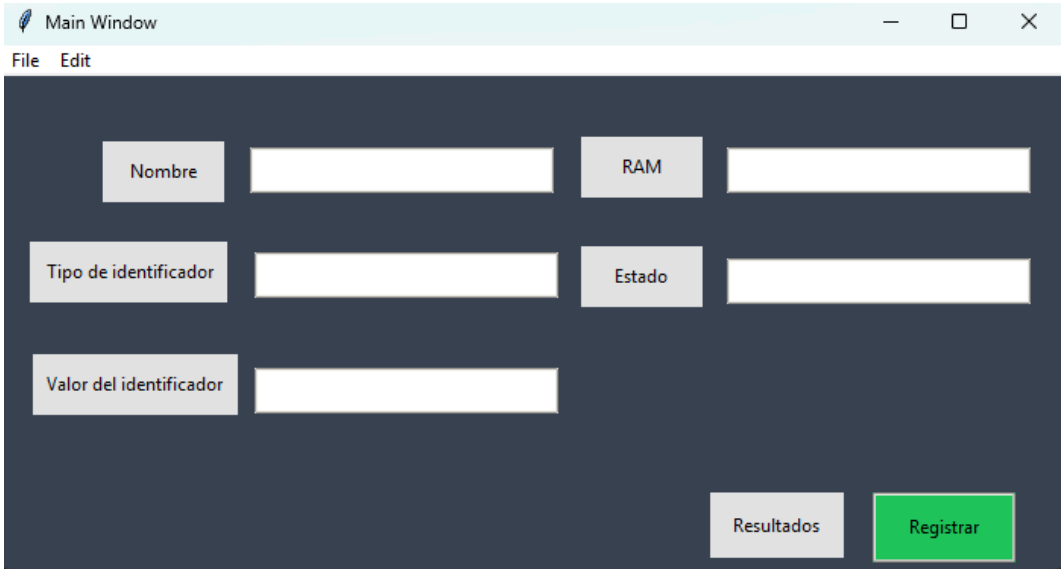
2. PARTE B - IDENTIFICACIÓN DE LOS CONCEPTOS

Pregunta	Respuesta del estudiante
¿Qué entidad representa la estructura equipo?	Representa los datos principales de un equipo
¿Cuáles son sus tres miembros/campos?	Nombre, ram y estado
Escribe una expresión que acceda al campo ram.	equipo["ram"]
¿Qué representa dato_alternativo?	Representa información extra de identificación, como el tipo y su valor
¿Qué regla del concepto de unión intenta modelar tipo + valor?	Modelar campos alternativos o variantes para un mismo elemento
Explica por qué esta representación no es una union nativa de memoria de C/C++.	Porque usa diccionarios de Python separados y no comparte el mismo espacio exacto de memoria física en el sistema

3. PARTE C - PREPARACIÓN DE PYUIBUILDER

No necesitas instalar PyUIBuilder para esta actividad; utiliza la versión web indicada por el profesor. Trabaja en Chrome o Edge y mantén tkinter como framework.

1. Abre y verifica que el framework seleccionado sea tkinter.
2. En la ventana principal coloca un Label y un Entry para cada dato: Nombre, RAM, Estado, Tipo de identificador y Valor del identificador.
3. Añade un Button con texto Registrar.
4. Añade un Label con texto Resultado:.
5. Acomoda la interfaz de forma clara. La apariencia no se evalúa por diseño avanzado; solo debe ser legible y funcional.
6. Antes de exportar, verifica que todos los controles estén colocados.
7. Usa Código de exportación para descargar el archivo .py. No uses la opción Exportar como clases.



Importante: completa el diseño antes de exportar. Después de modificar el archivo en VS Code, evita volver a exportar porque PyUIBuilder puede sobrescribir tus cambios manuales.

4. PARTE D - VERIFICACIÓN DEL CÓDIGO EXPORTADO

- 8. Abre el archivo .py exportado en Visual Studio Code.
- 9. Ejecuta el archivo sin modificarlo y comprueba que la ventana abre.
- 10. Localiza las líneas de los cinco Entry, el Button y el Label de Resultado.
- 11. Anota los nombres reales que PyUIBuilder asignó a esos widgets.

Widget	Nombre generado en tu código
Entry - Nombre	entradainput4
Entry - RAM	entradainput3
Entry - Estado	entradainput2
Entry - Tipo	entradainput
Entry - Valor	entradainput1
Button - Registrar	boton
Label - Resultado	label3

5. PARTE E - INTEGRACIÓN DE LA LÓGICA

Después del último widget creado y antes de `main.mainloop()`, añade una función siguiendo este modelo. Sustituye `entry`, `entry1`, etc. y `label_resultado` por los nombres reales de tu archivo.

```
def registrar_equipo():
    nombre = entry.get()
    ram = entry1.get()
    estado = entry2.get()
    tipo = entry3.get()
    valor = entry4.get()

    equipo = {
        "nombre": nombre,
        "ram": ram,
        "estado": estado
    }
    dato_alternativo = {"tipo": tipo, "valor": valor}

    texto = (
        f"{equipo['nombre']} | {equipo['ram']} GB | {equipo['estado']} | "
```

```

        f"{dato_alternativo['tipo']}: {dato_alternativo['valor']}"
    )
    label_resultado.config(text=texto)

    button.config(command=registrar_equipo)

```

Si el texto de resultado se corta, localiza el Label de resultado y realiza el mismo ajuste probado en clase:

```

label_resultado.configure(anchor="w")
label_resultado.place(x=..., y=..., width=300, height=40)

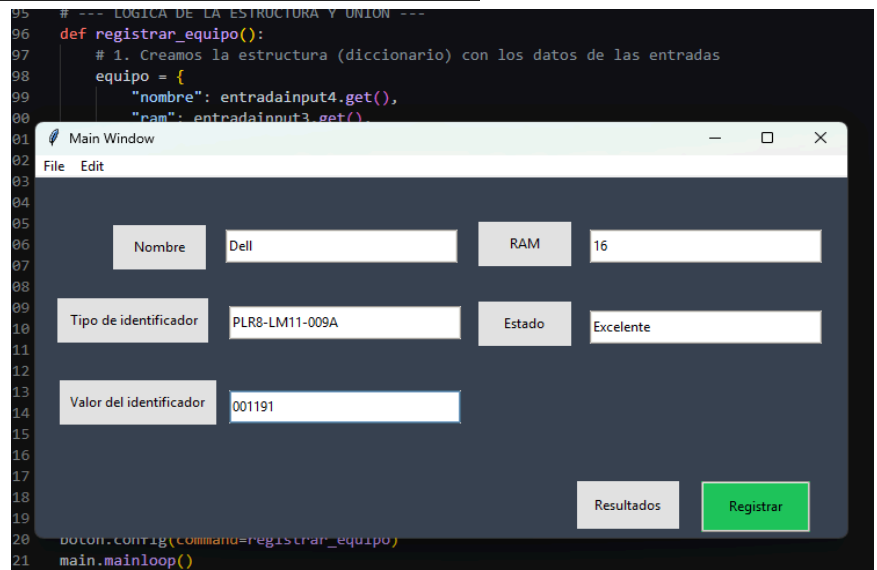
```

Conserva las coordenadas x e y que generó tu diseño; modifica principalmente anchor y width.

```

96 def registrar_equipo():
97     # 1. Creamos la estructura (diccionario) con los datos de las entradas
98     equipo = {
99         "nombre": entradainput4.get(),
100         "ram": entradainput3.get(),
101         "estado": entradainput2.get(),
102         # Simulamos la unión guardando el tipo y valor del identificador
103         "identificador": {
104             "tipo": entradainput.get(),
105             "valor": entradainput1.get()
106         }
107     }
108
109     # 2. Formateamos el texto que se mostrará en los resultados
110     resultado_texto = (
111         f"Nombre: {equipo['nombre']} | RAM: {equipo['ram']}GB | "
112         f"Estado: {equipo['estado']}\nId ({equipo['identificador']['tipo']}: "
113         f"{equipo['identificador']['valor']}"
114     )
115
116     # 3. Actualizamos la etiqueta de resultados (label3)
117     label3.config(text=resultado_texto)
118
119     # Conectamos la función al botón Registrar
120     boton.config(command=registrar_equipo)
121     main.mainloop()

```



- El label **resultado** se debió colocarlo más grande.

Nombre: PKOL-
Id (01192): Excel

6. PARTE F - PRUEBAS REQUERIDAS

Prueba	Qué debes comprobar	Resultado
1	La ventana abre sin errores.	La ventana se abre correctamente sin arrojar ningún error
2	Escribes todos los datos y el botón responde.	Funciona bien / El botón reacciona y ejecuta la acción al presionarlo
3	El resultado muestra nombre, RAM y estado.	El resultado muestra el nombre, la memoria RAM y el estado del equipo
4	El resultado muestra tipo y valor del identificador.	El tipo y el valor del identificador se visualizan perfectamente
5	El texto no se corta en el Label de resultado.	El texto se muestra completo y cabe de manera adecuada dentro de la etiqueta de resultado

7. EVIDENCIAS QUE DEBES SOMETER

- Archivo semana7_estructuras_uniones.py funcionando en consola.
- Archivo .py final de la interfaz exportada y modificada.
- Captura de la ejecución en consola.
- Captura de la interfaz mostrando un registro completo.
- Respuestas de la Parte B y tabla de nombres generados de la Parte D.
- Enlace al repositorio Git/GitHub o ubicación indicada por el profesor, si corresponde al flujo del curso.

8. PREGUNTAS DE CIERRE

- ¿Qué ventaja tuvo agrupar los datos del equipo en una sola estructura?
 - Ayudó a mantener toda la información junta y ordenada en un solo lugar, lo que hace que el código sea mucho más fácil de leer, entender y manejar en vez de tener muchas variables por todos lados.
- ¿Qué diferencia observas entre acceder a un miembro de la estructura y escribir directamente variables independientes?
 - La estructura se utilizó como un diccionario que conecta lógicamente los datos, mientras que las variables independientes están totalmente separadas y no guardan ninguna relación directa entre sí en el programa.
- ¿Qué representa la unión en este laboratorio y cuál es la limitación del modelo utilizado en Python?
 - Representa la oportunidad de guardar datos alternativos para un mismo fin. La gran limitación en Python es que se hace una simulación usando diccionarios y no es una unión real de memoria como en C o C++, por lo que los datos no comparten el mismo espacio físico en la computadora.
- ¿Qué cambió al pasar de la terminal a la interfaz? ¿Cambió la lógica o cambió principalmente la forma de entrada/salida?
 - El único cambio fue la forma de entrada y salida, Al iniciar el programa este abre una ventana con botones y cajitas de texto en lugar de un terminal común. El resultado va a ser el mismo.

9. LÍMITES Y POLÍTICA DE ENTREGA

No se requiere utilizar clases, objetos, constructores, herencia, bases de datos ni múltiples ventanas. Esos temas no forman parte del alcance oficial de la Semana 7.

Toda entrega posterior a la fecha límite tendrá una penalización inicial de 10 puntos. Por cada día adicional de atraso después del primer día fuera de término, se descontarán 5 puntos adicionales.